

1 POMDP with Infinite Action and States

スコアベースモデルを参考に信念更新の代わりに、次を学習する。

$$\begin{aligned} r_{0,0}, \dots, r_{0,K-1} &\sim \mathcal{N}(0, I) \\ \epsilon_{l,k} &\sim \mathcal{N}(0, I) \\ r_{lk} &= r_{l-1,k} + \alpha_l \mathbf{s}_t(r_{l-1,k}) + \sqrt{2\alpha_l} \epsilon_{l,k} \\ \mathbf{s}_{t,k} &= r_{Lk} \end{aligned}$$

実装では初期時刻専用の MLP を使い、初期スコアを次で定義する。

$$\mathbf{s}_0(s) = \nabla_s \left(g_{\theta_0}(o_0, s) - \frac{1}{2} \|s\|^2 \right), \quad g_{\theta_0}(o, s) = \text{MLP}_{\theta_0}(o, s).$$

ここで、 g_{θ_0} は通常時の観測 energy f_{θ_f} とパラメータを共有しない。 $-\|s\|^2/2$ は学習パラメータへの正則化ではなく、初期密度の裾と latent scale を固定する標準 Gaussian 基準密度の log density である。関数族を制限しない理想化では、この二次項を g_{θ_0} へ吸収した再 parameterization も可能である。しかし有限幅 MLP が非有界な latent 空間全体で同じ二次 tail を正確に表すとは限らず、実装上は normalizable な裾と scale を明示的に保証するため分離して残す。

$$\mathbf{s}_{t+1}(s') = \nabla_{s'} (f_{\theta_f}(o_{t+1}, s', a_t) + \text{logsumexp}_k(u_{\theta_u}(s', s_{t,k}, a_t)))$$

さらに、時刻 t での方策を次で定義。

$$a_t = \arg \max_a \pi_{\theta_\pi}(a | \mathbf{s}_t)$$

実装上、この argmax は離散方策の評価時にのみ用いる。学習時の離散行動は categorical 方策から sample する。連続行動では周辺密度の argmax を計算せず、DDPM reverse chain を sample して行動を決める。決定論的評価では初期 noise を 0 として各 reverse Gaussian の条件付き平均を順に通す `mean_chain` を使う。これは return 等の metric ではなく action readout の名前であり、周辺平均や MAP と等しいとは限らない。ここで、各関数は次のようにする。

$$\begin{aligned} f_{\theta_f}(o', s', a) &= \text{MLP}_{\theta_f}(o', s', a) \\ u_{\theta_u}(s', s, a) &= \text{MLP}_{\theta_u}(s', s, a) \\ \pi_{\theta_\pi}(a | \mathbf{s}_t) &= \text{Operator}_{\theta_\pi}[\mathbf{s}_t](a) \end{aligned}$$

ここで、アーキテクチャは以下とする。

$$\begin{aligned} &((s_{t,0}, \mathbf{s}_t(s_{t,0})), \dots, (s_{t,0}, \mathbf{s}_t(s_{t,K-1}))) \\ &\rightarrow \text{TransformerEncoder} \\ &\rightarrow \text{Pool}_k \\ &\rightarrow \text{MLP}(*, a) \end{aligned} \tag{1}$$

active 比較は `score_transformer` と GRU の 2 手法とする。連続 3 task では両手法へ同一の DiffusionPolicy を接続し、reverse step 数、noise schedule、variance floor、denoising advantage 重み、per-denoising-step PPO、`mean_chain` readout を共通にする。離散 CartPole でも同一構成の categorical head を使う。GRU の入力部分は観測と正規化した直前行動であり、後段の policy/value head は提案法と共通にする。

両手法で時間方向の `full` と `tbptt_1` を比較する。score belief の `full` は未終了 episode の保存 noise と入力 prefix を rollout 境界越しに再生し、GRU の `full` は観測・直前行動 prefix から hidden を episode 開始より再生する。`tbptt_1` はどちらも各環境 step へ入る belief/hidden だけを detach する。したがって各手法内では両 mode の forward 値は同一で、時間方向の gradient 範囲だけが異なる。

比較時は entropy 係数を 0 とする。production の GRU は入力 encoder 幅 [348, 185]、hidden 幅 176 とし、4 task での提案法との parameter 数の最大絶対相対差を 0.116% に合わせる。primary performance metric には、held-out episode における stochastic policy の最終 return を training seed 間で集約した値を用いる。数値診断として、初期 energy g_0 、観測 energy f 、遷移 energy u の pre-clip gradient norm、initial/recurisive score・particle norm、sample 数、non-finite 数をログへ保存する。

GAE の episode 終了 indicator d_t は、既定では真の終端と horizon 打ち切り (truncation) を区別せずどちらも 1 とし、次状態の bootstrap を止める。この規約は両手法・両 mode で完全に共通なので内部比較の公平性は保たれるが、絶対 return と value 学習は truncation-bootstrap 実装と直接比較できない。opt-in の設定 `ppo.bootstrap_on_truncation` を真にすると、truncation した step に限り $\gamma V(s_T)$ を GAE の delta 項へ加える (Pardo らの方式)。 s_T は auto-reset wrapper が捨てる打ち切り直後の真の観測である。既定は偽であり、値の異なる run を同一の集計へ混在させない。なお共通規約であることは手続き上の公平性を保証するだけで、打ち切り bias の負担が両表現で等しいことは意味しない。GRU の hidden は制約のない recurrent vector なので経過 step を暗黙に数えられるが、score belief の条件は状態上の事後粒子とその score だけで時間 channel を持たない。

Light-Dark の報酬は、遷移前の状態と行動に課金する二次コストと、終了 step の goal bonus B からなる。

$$r_t = - (0.5\|x_t\|^2 + 0.5\|u_t\|^2) + B \cdot \mathbf{1}\{\|x_{t+1}\| \leq 0.25\}$$

既定は $B = 0$ である。力学 $x' = x + u$ は決定論的なので事後平均は行動履歴の和だけで追跡でき、観測の価値は不確実性の縮小、すなわち goal 到達確率にしか現れない。 $B = 0$ ではその到達確率へ報酬が付かないため、production 条件 ($N = 5$ 、light at $x[0] = 5$ 、初期 $\mathcal{N}(2 \cdot \mathbf{1}, 0.5^2 I)$ 、horizon 30) では光へ寄る情報収集が厳密劣後する (実測で dead reckoning の平均 return が -31.8 、光で自己位置同定してから原点へ戻る方策が -87.9)。したがって既定パラメータの Light-Dark 結果を belief 品質の証拠として読んではならない。情報収集を最適化するには config の `goal_bonus` を損益分岐 (65–85) より十分大きい値 (目安 100 以上) へ設定するが、これは報酬定義そのものの変更であり、既定値で実行した既存 campaign とは混在させられない。

1. 言語は Python で OK

2. タスクは最低 3 種類以上はお願い。この実験内容に適したモデルでお願い。
3. 九大スパコンの genkai が使えます。そちらを実験環境としてください。ssh 接続用の config 情報はプロジェクト直下の ssh-config に書いてあります。
4. job の投下は絶対にしないで下さい。
5. なお行動の出力について、有限行動なら Softmax 出力ベース、無限次元行動なら拡散モデルベースを想定しています。
6. PPO での完全モデルフリー学習を想定しています。無限行動の際には適切に PPO 系のものをもってきてほしい。その内容についての tex は docs 内に用意しておいて。
7. 質問があれば教えて。
8. ついでに、関連研究をいくつかまとめてほしい。特に、今回の方策の作用素学習アーキテクチャは何かとかぶってるはずだから、調べてほしい。

コード管理について、

1. ソースコードはすべて ./src 内にお願いします。
2. まず学習実験を開始するスクリプトを作成し、超小規模・GPU なしでこの PC 上でプログラムが正しいことを確認してください。
3. その後、genkai-スパコン用の job ファイルを作成して下さい。こちらは学習開始のスクリプトを動かす、という内容にして簡略化して下さい。
4. job ファイルは DEBUG 用 job・本番用 job の 2 つを作成して下さい。Debug 用の job ファイルをユーザーが実行します。
5. ハイパーパラメータについては、超小規模 local モデル・本番用モデルの 2 つに分けて下さい。なお、genkai での DEBUG 用 job では本番用モデルを使用して下さい。
6. ここで、それぞれの設定値を、./config ディレクトリ内の json ファイルとしてまとめて下さい。
7. DEBUG 実行が終わったら、正しく動いていることの確認をお願いします。その後本番サーバーで実験を行います。
8. 実験結果はログ出力に加えて、csv, numpy 配列も作成し、保存しておいてください。また実験に際し保存して負い阿呆が良いものがあればそちらもお願いします。設定値なども保存しておいて下さい。
9. 最後に、実験結果のデータすべてをこちらの PC に持ってきて、./results ディレクトリ内において下さい。なお、実験の日時のタイムスタンプのサブディレクトリを作成し、その中に配置して下さい。